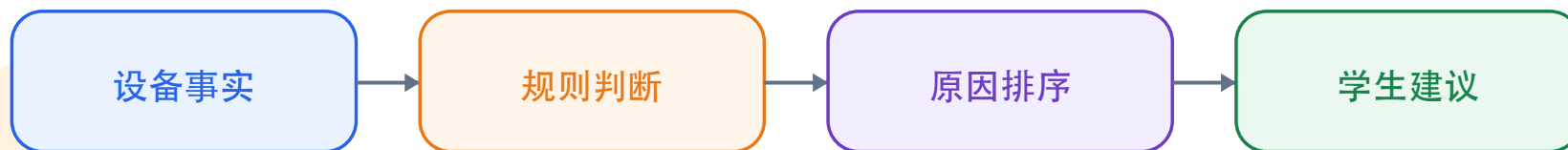


芯鉴知微

让嵌入式实验故障排查有证据、可解释、能复盘

面向高校嵌入式与物联网实验教学的智能诊断平台。

它不让AI凭空猜硬件故障，而是先采集真实设备事实，再用规则、故障树、结构化知识和受约束AI一步步形成建议。



依据：当前代码、12 份核心文档与团队架构图 · 适合项目介绍、组内评审和后续开发对齐

芯鉴知微解决的不是“知识搜索”，而是实验现场的诊断闭环。

学生遇到实验异常时，系统回答三个问题

发生了什么？

设备是否在线、哪项读数失败、日志出现了什么错误。

最可能为什么？

规则确定异常，故障树给出原因范围，AI只在范围内排序。

下一步怎么做？

结合实验规范与教师案例，给出逐级提示和排查步骤。

1 为什么现在不做 RAG

首阶段只有少量、明确的实验知识。实时采集、确定性规则和教师确认，比模糊相似度检索更重要。

2 为什么还需要 AI

真实证据往往分散在状态、日志和读数里。AI可以综合证据、排序候选原因，并把结果讲成学生听得懂的话。

3 为什么必须有教师

实验知识、案例结论和关键建议要经过教师审核。AI只能整理表达，不能把未经验证的猜测写进知识库。

先把几个专业词翻译成人话

02

这些词看起来复杂，本质上是在回答：材料放哪里、流程怎么走、结论怎样被约束。

DiagnosisContext

诊断材料包

一次诊断刚开始时收集到的现场材料：设备状态、日志、传感器值、实验类型等。

DiagnosisState

诊断过程记录本

在材料包基础上，持续记录规则结果、候选原因、AI排序、提示级别和学生反馈。每个节点都读写同一本“记录本”。

LangGraph

固定流程调度员

它按预先画好的流程图依次调用各节点，并根据“已解决/未解决/需教师帮助”走不同分支。它不是会自由行动的智能体。

Rule Engine

可复现的如果一那么

例如“DHT11连续读取失败20次”，规则就确定为SENSOR_READ_FAILED，并生成可核对的证据。

Fault Tree

原因范围清单

根据已确定的异常，列出允许考虑的原因，例如接线、GPIO配置或传感器本身；AI不能跳出这个范围。

Knowledge Context

推理前参考资料

把当前实验的规范、正常条件、标准故障映射和教师确认案例先交给AI，避免它脱离实验背景。

Knowledge Validation

推理后的门卫

AI排序后再次检查：证据ID是否真实、原因是否在候选集合、有没有违反实验规范或与规则冲突。

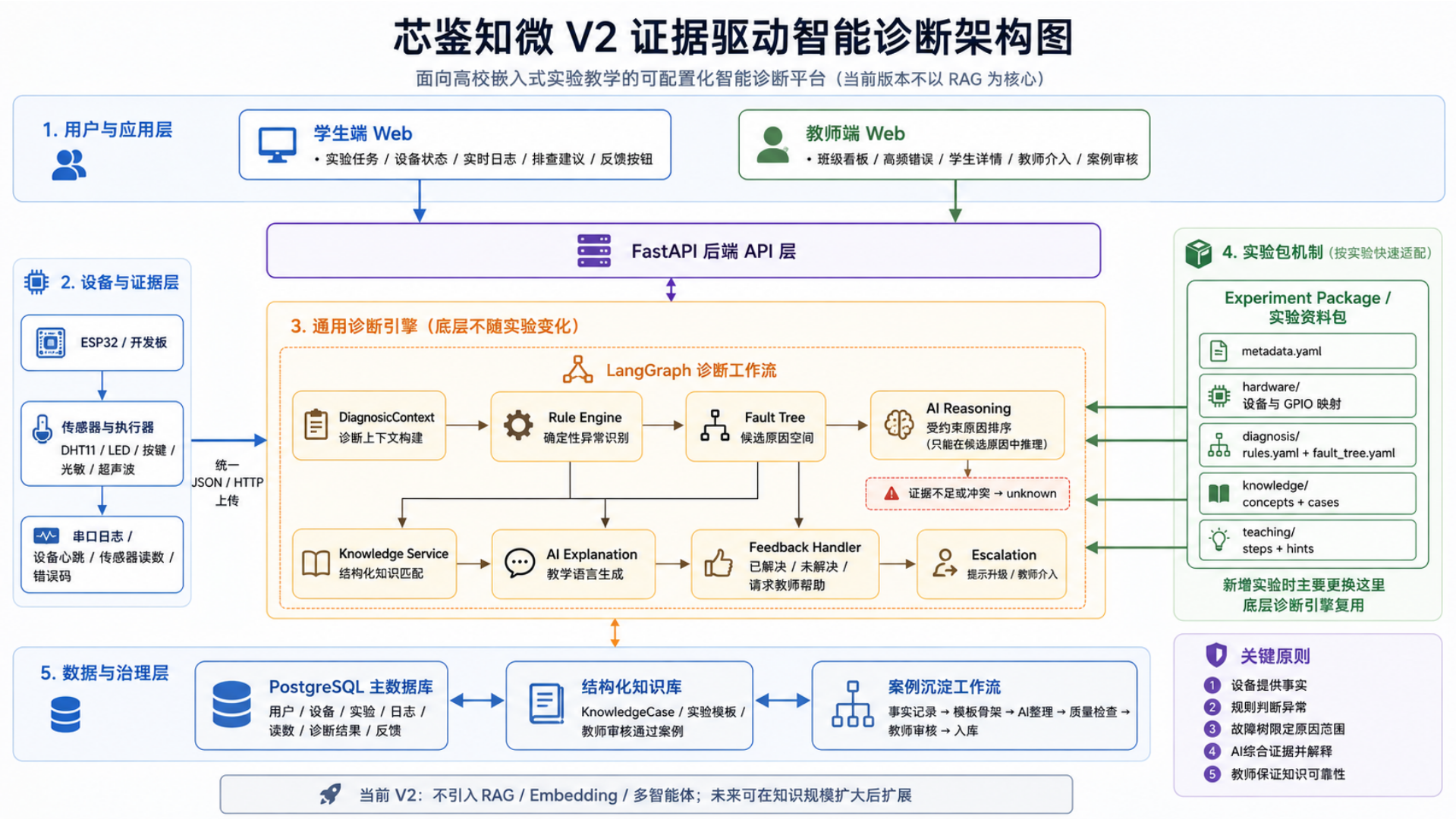
RAG

大规模知识查找助手

先从大量资料中检索相关内容，再交给AI。当前案例不多，结构化查询更稳，所以V2暂不把它放进核心链路。

系统全景：五层共同完成一次诊断

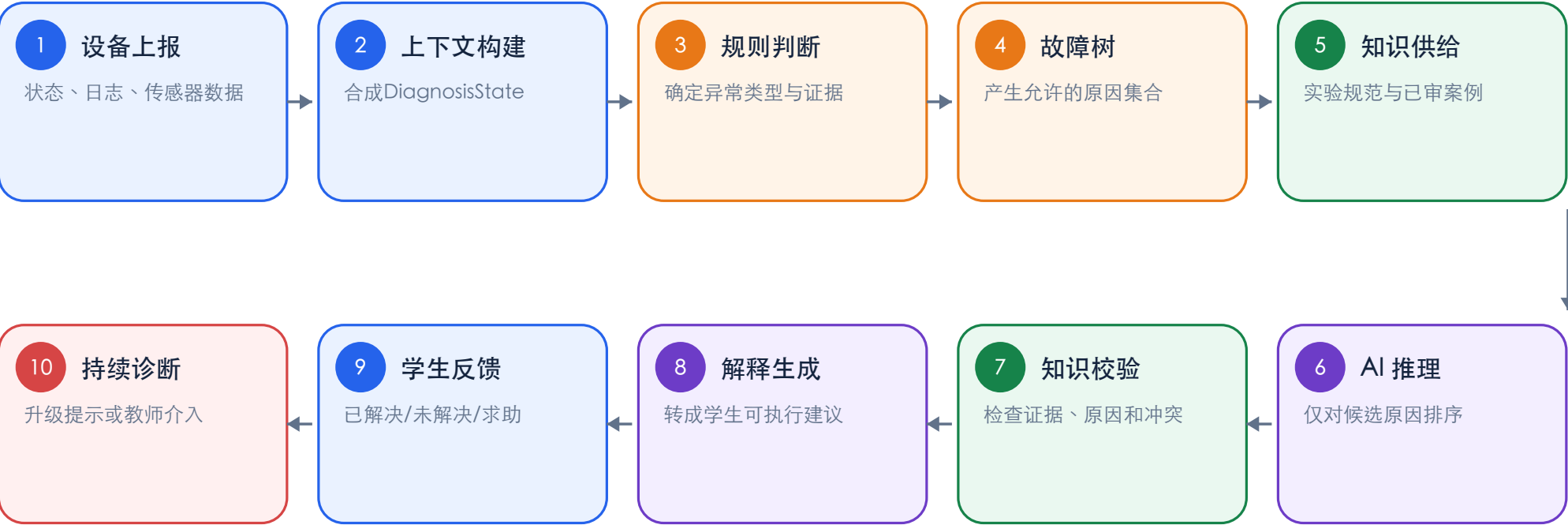
下图是团队详细架构图；当前实现已把知识服务进一步细分为“推理前供给 + 推理后校验”。



当前代码中的关键细化：故障树→知识上下文→AI原因排序→知识校验。也就是说，AI推理前有规范可参考，推理后还要再过一次硬校验。

一次诊断是怎样跑完的

LangGraph 只负责让节点按顺序执行和分支，不替代规则，也不让 Agent 自由规划。



持续诊断意味着什么？ 学生说“没解决”后，系统保留上一轮状态、增加失败次数、重新匹配知识与排序原因；达到条件时升级提示或交给教师，而不是从头乱猜。

每个模块只做自己擅长的事

稳定来自清楚分工：事实、判断、原因空间、经验、推理、审核彼此不能越权。

角色	核心职责	它实际做什么	明确不能做什么
设备	提供事实	在线状态、日志、传感器值、错误码、时间与环境	不能把现象直接说成故障原因
规则引擎	确定异常	用可复现条件产出error_type与证据	不能被AI覆盖
故障树	限定原因空间	给出与异常类型对应的候选原因	不能无限扩展原因
知识服务	供给与校验	提供规范、步骤、教师案例；验证AI输出	不是相似度搜索器
AI	综合与表达	排序候选原因、给置信度、说明依据、生成教学语言	不能发明事实或越出候选集合
教师	最终治理	审核知识、确认关键案例、处理升级请求	关键知识不能无人审核入库

一句底线：系统可以给出“最可能的原因”，但不能把概率判断包装成确定事实。证据不足时必须返回 unknown。

DiagnosisState：让一次诊断有记忆、有证据

06

所有节点读写同一份状态，既方便持续诊断，也方便回看“为什么得出这个结论”。

一次诊断保存什么

真实输入

device_status· logs· sensor_data· experiment_context

诊断结果

error_type· evidence· possible_causes·
knowledge_context

持续过程

hint_level· student_feedback· historical_failures

流程控制

need_teacher_help· diagnosis_status· state_revision

证据治理：不是“日志里看起来像”

- 设备上报采用统一JSON/HTTP协议，并用request_id保证重复上传不会重复记账。
- 原始数据与标准化数据分开保存，诊断可以追溯到原始事实。
- 规则生成的每条证据都落库，并取得真实UUID；AI只能引用这份允许列表中的证据ID。

为什么这样做

- 能解释：每个结论都能指出对应证据。
- 能复盘：学生反馈后仍知道上一轮发生了什么。
- 能审计：教师能区分设备事实、规则结论和AI推断。
- 能测试：模拟器可以稳定复现正常与故障场景。

实验包：换实验，不换诊断引擎

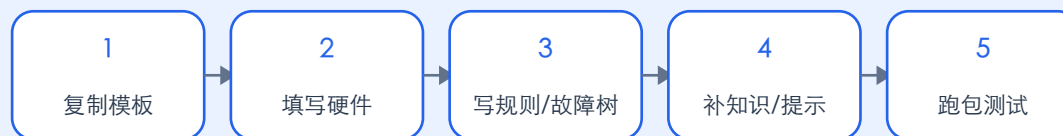
07

实验差异写在 YAML 资料里；通用引擎保持稳定，知识内容与 Python 代码分离。

一个实验包包含 10 类资料

- metadata.yaml 名称、版本、兼容信息
- hardware.yaml 设备与 GPIO 映射
- diagnosis/rules.yaml 确定性判断规则
- diagnosis/fault_tree.yaml 候选原因树
- knowledge/concepts.yaml 实验概念与规范
- knowledge/cases.yaml 已审案例
- teaching/steps.yaml 教学步骤
- teaching/hints.yaml 分级提示
- tests/normal_cases.yaml 正常场景
- tests/fault_cases.yaml 故障场景

新增实验时，底层引擎不应修改



当前完整示例

DHT11温湿度实验
LED实验

已有结构化案例

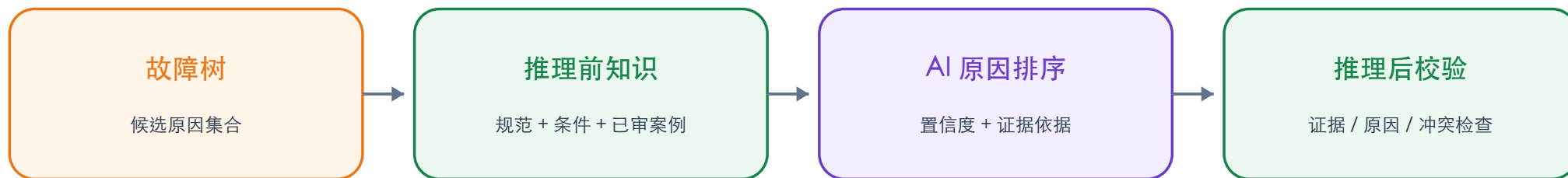
按键·光敏·超声波
仍需继续补齐正式实验包

重要说明：当前实验包与不少案例主要用于工程验证，并不等于已经通过真实硬件和教师正式验收。

AI 可以推理，但必须戴着“护栏”

08

AI 的价值不是替代硬件专家，而是把分散证据组合成受约束、可读的判断。



AI 被允许做的事

- 只在故障树给出的候选原因中排序。
- 综合多个真实证据，并给出0~1的置信度。
- 说明为什么把某个原因排在前面。
- 把已有诊断结果转成学生能理解、能执行的语言。
- 证据不够或相互冲突时，明确返回unknown。
-

AI 被禁止做的事

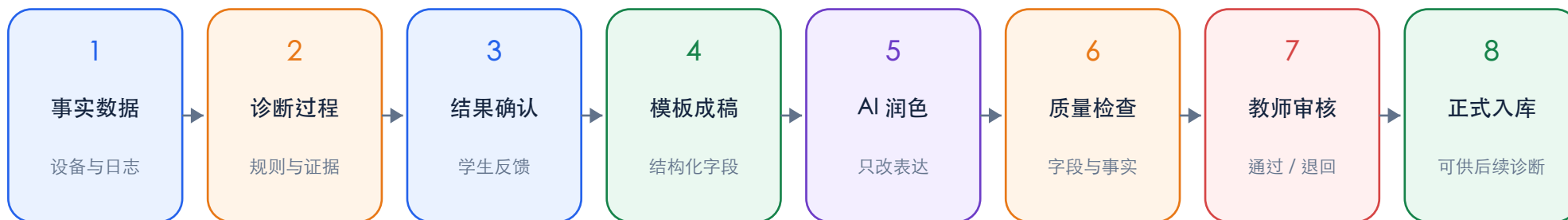
- 不能提出候选集合之外的新故障。
- 不能引用数据库中不存在的evidence_id。
- 不能覆盖规则已经确定的error_type。
- 不能把未经验证的表达当成设备事实。
- 不能自动把自己生成的案例写入正式知识库。
-

校验失败怎么办：丢弃越权部分 → 回退到确定性结果 / unknown → 保留审计记录 → 必要时请求教师帮助。

知识不是 AI 自动“长出来”的

09

正式知识必须从事实出发，经质量检查和教师审核；AI 只负责整理表达。



KnowledgeCase 保存哪些内容

experiment· errorType· symptom· evidence·
possibleCauses· solutionSteps· teacherNotes·
reviewStatus

进入正式知识库的条件

- 事实 and 证据可追溯。
- 原因与最终解决结果一致。
- 表达不添加未经确认的信息。
- 教师明确审核通过。

这样沉淀出来的知识，既能帮助下一位学生，也能明确知道“是谁、根据什么、在什么时候确认的”。

学生和教师看到的是同一个闭环的两面

学生获得可执行帮助；教师看到高频问题、升级请求和可审核案例。

学生端

- 1 看现场
设备在线状态、实时日志、传感器趋势
- 2 看诊断
异常类型、证据、原因排序、排查步骤
- 3 按提示操作
从轻提示到明确步骤，避免直接给答案
- 4 反馈结果
已解决/未解决/请求教师帮助

反馈与介入



教师端

- 1 看班级
异常分布、重复失败、高频错误
- 2 看学生
某次诊断的事实、证据、步骤和反馈
- 3 做介入
处理求助、纠正结论、补充教学建议
- 4 审知识
审核候选案例，决定是否进入正式知识库

“DiagnosisState 全流程可视化” 直白说：把这次诊断走过的每一步、用了什么证据、为什么升级提示，按时间顺序展示出来。

现在已经做到哪一步

下面区分“工程功能已打通”和“真实教学效果已验证”，避免把模拟测试说成现场成熟。

已打通的工程能力

- 设备数据上传、重试去重、原始数据与标准化数据保存。
- 规则诊断、故障树、两阶段知识服务、受约束AI推理。
- LangGraph状态工作流、学生反馈、提示升级与教师介入。
- DHT11/LED实验包示例，以及结构化知识案例和案例审核流程。
- 学生端、教师端、FastAPI接口、PostgreSQL与Docker启动路径。
-

最近一次全量验证

183

后端测试通过

16

设备模拟器测试

26

前端单元测试

5

浏览器端到端测试

后端

FastAPI模块化单体
LangGraph+PostgreSQL

前端

Vue 3+TypeScript
学生端/教师端

运行

Docker Compose
模拟器可复现故障

AI

Provider默认关闭
可回退确定性诊断

演示建议：先启动数据库和后端 → 运行设备模拟器 → 打开学生端看诊断 → 用教师端处理升级与审核案例。

下一步：先把真实证据做扎实，再谈规模化检索

12

当前版本适合工程联调与方案验证；真实教学落地还需要硬件、知识和课堂数据的共同验收。

现在不能夸大的地方

- 合成数据通过，不等于真实硬件诊断准确。
- 实验包示例通过，不等于教师已正式签字认可。
- 知识校验能防越权，但不是完整的电气安全证明。
- 学生反馈后会继续推理，但目前不会自动重新采集设备上下文。
- 版本兼容、并发锁、知识发布不可变性仍需进一步强化。

推荐推进顺序

- 1 真实硬件基线**
BOM、接线、GPIO、电源、固件版本
- 2 真实故障样本**
按证据标准采集并由教师标注
- 3 课堂试点**
验证准确率、解释质量与教师工作量
- 4 生产强化**
权限、隐私、预算、并发、可观测性
- 5 按需引入 RAG**
案例规模扩大且结构化查询不足时再做

未来 RAG 怎么接，才不会破坏现在的可靠性

只对“教师已审核”的知识建立派生向量索引；RAG只扩展知识召回，不改变规则结果、故障树候选集合、证据ID校验和教师审核链。结构化知识库仍是真相来源。

开发准则：小步兼容 · 证据优先 · 规则不可覆盖 · 内容配置化 · 教师可审核 · 测试和文档同步更新